11.2.1 Serializing objects

Returning simple strings isn't much fun and is of limited value. Thankfully, Ninja serializes more interesting objects. Most native Python data types can be returned from a view, and Ninja will turn them into JSON. There are some restrictions, but they're the same restrictions imposed by Python's `json` library. This means lists of strings, or a good old dictionary, can be returned from an API view.

One of the exercises in chapter 2 had you add a view that showed your project's version information. In it, you returned a `JsonReponse` object that serialized a dictionary. The following listing shows the Ninja equivalent. Add it to RiffMates/home/api.py.

Listing 11.3 You can serialize a `dict` from a Ninja API endpoint

```
# RiffMates/home/api.py
...
@router.get("/version/")
def version(request):
    data = {
        "version": "0.0.1",
    }
    return data
```

Declare a function that handles a GET call for the URL ending in /version/.

Anything the json module can serialize can be returned through Ninja.

Note the URL argument to `@router.get()` ends with a slash (`/`). You'll recall from chapter 8 that Django redirects any URLs not ending in a slash to their equivalent with a slash on the end. This is actually my only real complaint about Ninja—it doesn't handle this well. You can create API endpoints without the slash, but I've run into problems with them: there are cases where the redirect will trigger and others where it won't. Rather than trying to figure out which is which, always declare the URL ending in a slash and avoid the problem altogether.

Since the `version` API endpoint uses the existing `router` object in the file, and that router already got registered with Ninja, you have nothing else to do. Your API now supports two endpoints. Make sure your development server is running, and visit http://localhost:8000/api/v1/home/version/ to try it out.

As mentioned earlier in the chapter, the most likely thing you'll do with your API is expose your ORM Model classes. Django Model classes aren't compatible with Python's JSON serializer, but Ninja provides tools for converting your Model data to JSON with very little work.

Ninja is built on top of another third-party library, called Pydantic. This library is similar to the Django ORM, in that it provides tools for declaring fields in classes, but unlike the ORM, it uses Python type hints to do so. Ninja extends Pydantic's object model with a class called `Schema`. You can declare your own serializable objects by inheriting from `Schema` and declaring fields. If you've ever used Python's dataclasses module, it is similar. Pydantic has been around longer than dataclasses, which is why there are multiple ways of doing things now.